Farmer–Toda Discretization

MATLAB and Python User Guide

Alexis Akira Toda*

First version: June 2, 2019
Updated: October 26, 2026

Abstract

This guide documents the Farmer–Toda routines for approximating continuous distributions and nonlinear Markov processes by finite-state models that match selected moments. The repository provides a namespaced MATLAB package and a PYTHON implementation with common model coverage, automated tests, and cross-language numerical reference checks. The guide describes installation, the public application programming interface (API), model-specific inputs and outputs, grid and moment choices, numerical safeguards, compatibility names, and the construction of new discretization routines.

Keywords: discretization; Markov chain approximation; maximum entropy; quadrature; moment matching

Contents

1 Introduction **2**

2 Getting started **3**

2.1 MATLAB 3

2.2 Python 3

2.3 Output convention 4

3 Public API **4**

4 Maximum-entropy moment matching **4**

*Department of Economics, Emory University. Email: alexis.akira.toda@emory.edu.

5 Process discretization routines **5**
5.1 Gaussian AR(1) . . . 5
5.2 Gaussian VAR(1) . . . 6
5.3 AR(p) with Gaussian-mixture shocks . . . 6
5.4 AR(1) with stochastic volatility . . . 7
5.5 Cox–Ingersoll–Ross process . . . 8
5.6 Ornstein–Uhlenbeck processes . . . 9

6 Distribution and quadrature routines **9**
6.1 Distribution from centered moments . . . 9
6.2 Gaussian-mixture quadrature . . . 10
6.3 Data-driven Gaussian quadrature . . . 10

7 Choosing grids and moments **10**

8 Writing a custom discretization routine **11**

9 Numerical safeguards and verification **12**
9.1 Default change in version 0.2.0 . . . 12

10 Compatibility names **13**

11 Citation and software responsibility **14**

1 Introduction

The `discretization` repository implements the moment-preserving discretization method developed by Farmer and Toda (2017), building on the maximum-entropy construction of Tanaka and Toda (2013) and the convergence analysis in Tanaka and Toda (2015). The central idea is to start from a finite grid and a prior distribution, then adjust the probability weights so that selected moments agree with their targets.

The modernized repository has two user-facing implementations:

- a MATLAB package called `discretization`, invoked through names such as `discretization.discreteAR`; and
- a PYTHON package with snake-case names such as `discrete_ar`.

Historical MATLAB entry points remain available as compatibility wrappers, but new code should use the package-qualified names in this guide. The two implementations are tested against deterministic MATLAB R2024b reference outputs. Numerical solvers

can produce small platform-dependent differences, so parity is assessed with explicit tolerances rather than bit-for-bit identity.

Whenever these routines are used in research, please cite at least Farmer and Toda (2017); citations to Tanaka and Toda (2013) and Tanaka and Toda (2015) are also appropriate when the maximum-entropy construction is material to the application.

2 Getting started

2.1 MATLAB

Add only the repository root to the MATLAB path. Package and private helper resolution then occur automatically.

```
addpath('path/to/discretization-master')

[P, X] = discretization.discreteAR(0, 0.9, 0.1, 9);
```

The implementation uses optimization and probability-distribution routines, including `fminunc`, `fmincon`, `lsqnonneg`, `normpdf`, and `norminv`. A standard installation therefore requires the Optimization Toolbox and the Statistics and Machine Learning Toolbox.

2.2 Python

Install the package from the repository's `python` directory. Python 3.10 or newer, NumPy, and SciPy are required.

```
python -m pip install ./python
```

```
from discretization import discrete_ar

P, grid = discrete_ar(
    mean=0.0,
    persistence=0.9,
    innovation_std=0.1,
    state_count=9,
)
```

2.3 Output convention

All process routines return a row-stochastic transition matrix P. Entry P_{ij} is the probability of moving from current state i to destination state j. A univariate PYTHON grid is one-dimensional. Multivariate and joint-state grids use one row per state variable and one column per Markov state. In MATLAB, process grids follow the same row-by-variable convention.

Tensor-product states are ordered with the last state dimension changing fastest. This detail matters when a transition matrix is combined with a value function, simulation code, or another tensor-product operator.

3 Public API

Table 1 lists the main public functions. The long descriptive names are intentional: they distinguish an autoregression with Gaussian-mixture innovations from a mixture autoregression, and they identify the particular stochastic-volatility model being approximated.

Table 1: Canonical public API

Purpose	MATLAB	PYTHON
Gaussian AR(1)	`discretization.discreteAR`	`discrete_ar`
Gaussian VAR(1)	`discretization.discreteVAR`	`discrete_var`
Cox–Ingersoll–Ross	`discretization.discreteCIR`	`discrete_cir`
AR with mixture shocks	`discretization.discreteGaussianMixtureAR`	`discrete_gaussian_mixture_ar`
AR with stochastic volatility	`discretization.discreteStochasticVolatilityAR`	`discrete_stochastic_volatility_ar`
Distribution from moments	`discretization.momentMatchedDistribution`	`moment_matched_distribution`
Maximum-entropy weights	`discretization.maximumEntropyWeights`	`maximum_entropy_weights`
Gaussian-mixture quadrature	`discretization.gaussianMixtureQuadrature`	`gaussian_mixture_quadrature`
Data-driven quadrature	`discretization.dataDrivenGaussianQuadrature`	`data_driven_gaussian_quadrature`
CIR transition density	`discretization.cirTransitionDensity`	`cir_transition_density`

4 Maximum-entropy moment matching

Let $D = (x_1, \ldots, x_N)$ be a fixed grid, let $q_n \geq 0$ be prior weights, and let $T(x_n) \in \mathbb{R}^L$ be the moment functions. The target moment vector is $\overline{T} \in \mathbb{R}^L$. The routines

minimize Kullback–Leibler divergence from the prior subject to the probability and moment restrictions. The dual representation gives

$$p_n(\lambda) = \frac{q_n \exp\{\lambda^\mathsf{T}[T(x_n) - \overline{T}]\}}{\sum_{j=1}^{N} q_j \exp\{\lambda^\mathsf{T}[T(x_j) - \overline{T}]\}}. \tag{1}$$

The gradient of the log normalizer is the moment error. The Hessian is the weighted covariance matrix of the moment functions.

The modernized implementation evaluates the dual objective with a log-sum-exp shift, supplies analytic first and second derivatives, caches solver settings, and retains the best finite candidate across solver attempts. Before expensive higher-order solves, selected routines test whether the target lies in the convex hull generated by the finite grid. An infeasible target cannot be matched by any probability vector on that grid.

The low-level interfaces are

```
[p, lambda, momentError] = ...
    discretization.maximumEntropyWeights(D, T, TBar, q, lambda0);
```

and

```
p, dual, moment_error = maximum_entropy_weights(
    grid, moment_function, target_moments, prior, dual0
)
```

where `q`/`prior` and `lambda0`/`dual0` are optional. Grid points occupy columns in MATLAB and the last array axis in PYTHON.

5 Process discretization routines

5.1 Gaussian AR(1)

The Gaussian autoregression is

$$x_t = (1 - \rho)\mu + \rho x_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim N(0, \sigma^2). \tag{2}$$

The canonical calls are

```
[P, X] = discretization.discreteAR( ...
    mu, rho, sigma, Nm, method, nMoments, nSigmas);
```

```
P, grid = discrete_ar(
```

```
    mean, persistence, innovation_std, state_count,
    method="even", moment_count=2, grid_width=None,
)
```

The default is an evenly spaced grid and two conditional moments. The MATLAB routine also exposes its quadrature-grid variants. The current PYTHON `discrete_ar` interface supports the evenly spaced grid.

5.2 Gaussian VAR(1)

For

$$y_t = b + By_{t-1} + \eta_t, \qquad \eta_t \sim N(0, \Psi), \tag{3}$$

use

```
[P, X] = discretization.discreteVAR( ...
    b, B, Psi, Nm, nMoments, method, nSigmas);
```

```
P, grid = discrete_var(
    constant, lag, innovation_covariance, state_count,
    moment_count=2, method="even", grid_width=None,
)
```

Here `Nm`/`state_count` is the number of grid points in each dimension, so a d-dimensional VAR has N_m^d states and an $N_m^d \times N_m^d$ transition matrix. The covariance matrix Ψ must be positive definite and the spectral radius of B must be below one.

Available methods are `'even'`, `'quadrature'` (Gauss–Hermite), and `'quantile'`. The quantile method is retained for replication but is generally poor. For a weakly persistent process, quadrature can be accurate; for persistent processes, the evenly spaced grid is usually more reliable. The implementation rotates standardized innovations to make unconditional component variances as similar as possible. The two-dimensional rotation has a closed-form solution.

5.3 AR(p) with Gaussian-mixture shocks

Consider

$$x_t - \mu = \sum_{j=1}^{p} a_j (x_{t-j} - \mu) + \varepsilon_t, \tag{4}$$

where ε_t is an i.i.d. Gaussian mixture. The interfaces are

```
[P, X] = discretization.discreteGaussianMixtureAR( ...
```

```
    mu, A, pC, muC, sigmaC, Nm, nMoments, method, nSigmas);
```

```
P, grid = discrete_gaussian_mixture_ar(
    mean, ar_coefficients, mixture_probabilities,
    mixture_means, mixture_standard_deviations, state_count,
    moment_count=2, method="even", grid_width=None,
)
```

The vectors `pC`, `muC`, and `sigmaC` contain component probabilities, means, and standard deviations. Probabilities must be nonnegative and sum to one. The AR companion matrix must be stable.

Grid methods are evenly spaced, Gauss–Legendre, Clenshaw–Curtis, Gauss–Hermite, and Gaussian-mixture quadrature. MATLAB method strings use hyphens and `'GMQ'`; PYTHON uses `'gauss_legendre'`, `'clenshaw_curtis'`, `'gauss_hermite'`, and `'gaussian_mixture'`, while accepting the corresponding hyphenated aliases. Unless the process is weakly persistent and a quadrature prior is especially useful, the evenly spaced grid is the safest default. The number of states grows as N_m^p, so small p is essential.

5.4 AR(1) with stochastic volatility

The joint model is

$$y_t = \lambda y_{t-1} + u_t, \qquad u_t \sim N(0, \mathrm{e}^{x_t}), \tag{5}$$

$$x_t = (1-\rho)\mu + \rho x_{t-1} + \epsilon_t, \qquad \epsilon_t \sim N(0, \sigma_E^2). \tag{6}$$

Use

```
[P, yxGrid] = discretization.discreteStochasticVolatilityAR( ...
    lambda, rho, sigmaU, sigmaE, Ny, Nx, method, nSigmaY);
```

```
P, joint_grid = discrete_stochastic_volatility_ar(
    persistence, volatility_persistence,
    unconditional_innovation_std, volatility_innovation_std,
    state_count, volatility_state_count,
    method="even", grid_width=None,
)
```

The returned grid has the y state first and log variance second. There are $N_y N_x$

joint states. The argument `sigmaU` is the unconditional standard deviation used to calibrate the innovation scale; `sigmaE` is the standard deviation of the log-variance innovation. The volatility process uses the same grid choices as `discreteVAR`. An evenly spaced grid is recommended when ρ is large.

5.5 Cox–Ingersoll–Ross process

For the CIR diffusion

$$\mathrm{d}r_t = a(b - r_t)\,\mathrm{d}t + \sigma\sqrt{r_t}\,\mathrm{d}W_t, \tag{7}$$

the transition density can be written

$$f(r'; r) = c\mathrm{e}^{-u-v}\left(\frac{v}{u}\right)^{q/2} I_q(2\sqrt{uv}), \tag{8}$$

where

$$c = \frac{2a}{(1-\mathrm{e}^{-a\Delta})\sigma^2}, \quad q = \frac{2ab}{\sigma^2} - 1, \quad u = cr\mathrm{e}^{-a\Delta}, \quad v = cr'.$$

The Feller restriction $\sigma^2 \le 2ab$ is imposed. The conditional moments are

$$\begin{aligned}
\mathrm{E}[r'|r] &= r\mathrm{e}^{-a\Delta} + b(1-\mathrm{e}^{-a\Delta}),\\
\mathrm{Var}[r'|r] &= \frac{\sigma^2}{a}(1-\mathrm{e}^{-a\Delta})\left(r\mathrm{e}^{-a\Delta} + \frac{b}{2}(1-\mathrm{e}^{-a\Delta})\right).
\end{aligned}$$

The interfaces are

```
[P, X] = discretization.discreteCIR( ...
    a, b, sigma, Delta, N);
```

```
P, grid = discrete_cir(
    mean_reversion, long_run_mean, volatility, step,
    state_count,
)
```

Both languages automatically select an evenly spaced grid that satisfies the sufficient conditions for matching the first two conditional moments and maximizes stationary coverage within the feasible set. Python retains its old grid construction only when coverage or method is supplied explicitly. The density itself is available through `cirTransitionDensity` and `cir_transition_density`.

5.6 Ornstein–Uhlenbeck processes

The centered Gaussian OU construction matches the exact conditional mean and variance of $\mathrm{d}X_t = -\kappa X_t\,\mathrm{d}t + \sigma\,\mathrm{d}W_t$. With $\omega = \sigma/\sqrt{2\kappa}$, its automatic grid has endpoints $\pm\omega\sqrt{N-1}$. A fixed increasing grid can instead be supplied; infeasible two-moment grids raise an error before optimization of any row.

```
[P,X] = discretization.discreteOU(.7,.3,.25,21);
[PJ,XJ,info] = discretization.discreteOUAsymmetricLaplaceJumps( ...
    .7,.3,.1,2.7,1.05,.25,21);
```

```
P, grid = discrete_ou(.7, .3, .25, 21)
PJ, grid_j = discrete_ou_asymmetric_laplace_jumps(
    .7, .3, .1, 2.7, 1.05, .25, 21)
```

The jump routine uses a centered geometric aggregate of asymmetric-Laplace jumps at the end of each step. With $q = \exp(-\lambda\Delta)$, the count law is $\Pr(M = m) = q(1-q)^m$. Conditional on a positive count, aggregate rates satisfy $a_\Delta b_\Delta = qab$ and $a_\Delta - b_\Delta = a - b$. The Gaussian-plus-jump reference density is evaluated in log space, and both conditional moments are matched. Zero intensity recovers Gaussian OU. At finite step size this is a splitting approximation to compensated compound-Poisson OU; decrease Δ to check timing error and use powers of the matrix for longer horizons. The third MATLAB output, or Python `return_diagnostics=True`, reports step parameters and conditional variance. See `API.md` for signatures.

6 Distribution and quadrature routines

6.1 Distribution from centered moments

To construct an N-point distribution from centered moments, use

```
[x, p] = discretization.momentMatchedDistribution( ...
    N, cMoments, q);
```

```
grid, probability = moment_matched_distribution(
    state_count, centered_moments, prior=None
)
```

The first centered moment must be zero and the second must be positive. If the prior is omitted, a Gaussian prior is used. The grid spans $\sqrt{2N}$ standard deviations in each

direction, as recommended by Farmer and Toda (2017).

6.2 Gaussian-mixture quadrature

The functions `gaussianMixtureQuadrature` and `gaussian_mixture_quadrature` compute Gaussian-quadrature nodes and weights for a Gaussian mixture using the Golub–Welsch construction. A single-component input recovers ordinary Gaussian quadrature.

6.3 Data-driven Gaussian quadrature

The entry points are

- MATLAB: `dataDrivenGaussianQuadrature`; and
- PYTHON: `data_driven_gaussian_quadrature`.

Both functions construct quadrature nodes directly from one-dimensional data. This method is separate from maximum-entropy moment matching and is based on Toda (2021).

An N-point rule requires moments through order $2N$. For positive or heavy- tailed data, apply the method to logged observations when appropriate and exponentiate the resulting nodes. This transformation reduces sensitivity to high-order raw moments.

7 Choosing grids and moments

- **Start with two moments.** Matching the conditional mean and variance usually provides a good balance between accuracy and feasibility. Higher moments require more grid support and can be infeasible near grid boundaries.
- **Use an evenly spaced grid for persistent processes.** A wider grid is often more valuable than a high-order quadrature rule when the conditional distribution shifts substantially across current states.
- **Use quadrature for weak persistence.** Gauss–Hermite or a model-specific quadrature prior can be effective when conditional means stay close to the center of the unconditional grid.
- **Avoid the VAR quantile grid for new work.** It is retained for replication and comparison.

- **Respect dimensional growth.** With N points in each of d dimensions, the transition matrix contains N^{2d} entries. Dense methods become impractical quickly even when each one-dimensional solve is cheap.

When a warning reports that k moments could not be matched, the routine uses the highest lower-order match that succeeded. The usual remedies are to widen the grid, increase the number of points, or request fewer moments. Suppressing the warning without checking the resulting moment errors is not recommended.

8 Writing a custom discretization routine

A one-dimensional custom routine has four steps:

(i) choose grid points and quadrature/integration weights;

(ii) compute a strictly positive prior on that grid for each current state;

(iii) define scaled conditional moment functions and targets; and

(iv) call the maximum-entropy weight routine, checking the returned moment error and providing a lower-order fallback when needed.

For example, the following code constructs weights on a fixed grid with mean zero and variance one.

```
D = linspace(-4, 4, 9);
T = @(x) [x; x.^2];
TBar = [0; 1];
q = normpdf(D);

[p, lambda, momentError] = ...
    discretization.maximumEntropyWeights(D, T, TBar, q);
```

The equivalent PYTHON call is

```
import numpy as np
from scipy.stats import norm
from discretization import maximum_entropy_weights

grid = np.linspace(-4, 4, 9)
moment_function = lambda x: np.vstack((x, x**2))
targets = np.array([0.0, 1.0])
prior = norm.pdf(grid)
```

```
probability, dual, moment_error = maximum_entropy_weights(
    grid, moment_function, targets, prior
)
```

Scaling moment functions by a representative grid magnitude generally improves conditioning. Prior weights need not sum to one, but they must be nonnegative with at least one positive entry. Process reference weights are evaluated in log space: subtract the row maximum, clamp only at log(realmin), then exponentiate and normalize. This preserves rare transitions without an absolute density floor. The generic maximum-entropy routine preserves the support of user-supplied priors.

9 Numerical safeguards and verification

9.1 Default change in version 0.2.0

Version 0.2.0 removes the absolute 10^{-8} reference-weight floor from the AR, VAR, Gaussian-mixture AR and stochastic-volatility routines in both languages. Default matrices therefore change. Artificial tail mass can cause material accumulated errors in likelihood-based estimation even when simulated moments are accurate. Historical grid widths and other process defaults remain. To recover the 0.1.0 process algorithms, use

```
[Pold,X] = discretization.discreteAR(0,0.9,0.1,81, ...
    'ProbabilityFloor',1e-8);
```

```
Pold, grid = discrete_ar(0, .9, .1, 81,
                         probability_floor=1e-8)
```

The floor applies to reference weights, not final probabilities. Explicit positive floors select the legacy algorithm and return no matched-moment counts. Default MATLAB calls return diagnostics as a third output; Python uses `return_diagnostics=True`. Counts are reported by row, and by rotated dimension for VAR, with standardized residual tolerance 10^{-7}.

The default matcher proceeds sequentially from the mean upward and retains the highest successful count. Centered standardized nodes z must satisfy $\min z < 0 < \max z$ for strict mean feasibility and $-\min z \max z \geq 1$ for unit variance; a convex-hull check also enforces grid-spacing and higher-moment restrictions. An infeasible or numerically failed mean match gives a point mass nearest the conditional

mean and a count of zero. A higher-moment failure retains the preceding match. Near a truncated boundary, exact moments require tilting the normalized Gaussian density. Check both grid width and resolution before using a chain for likelihoods.

The modernized code includes the following safeguards:

- log-sum-exp evaluation of the dual objective;
- analytic gradient and Hessian calculations;
- finite-candidate fallback when an optimizer fails;
- convex-hull feasibility checks for selected moment targets;
- relative log-reference stabilization without an absolute floor;
- direct linear solves instead of explicit matrix inverses;
- a closed-form two-dimensional VAR variance rotation; and
- reduced temporary allocation in tensor-product transition matrices.

Automated tests cover row stochasticity, nonnegativity, default arguments, historical compatibility wrappers, alternate grid methods, higher-dimensional VAR structure, and reference parity between MATLAB and PYTHON. The new default reference files were generated with MATLAB R2026a; the historical R2024b files are retained for legacy regression checks. Representative performance workloads are provided in the repository's `benchmarks` directory; measured times are machine-specific.

10 Compatibility names

Table 2 maps historical MATLAB calls to the canonical API. The old entry points forward to the package implementation and issue migration warnings. They remain available during modernization so that replication code does not break unexpectedly.

Table 2: MATLAB compatibility-name migration

Historical name	Canonical replacement
`discreteAR`	`discretization.discreteAR`
`discreteVAR`	`discretization.discreteVAR`
`discreteCIR`	`discretization.discreteCIR`
`discreteGMAR`	`discretization.discreteGaussianMixtureAR`
`discreteSV`	`discretization.discreteStochasticVolatilityAR`
`discreteNP`	`discretization.momentMatchedDistribution`
`discreteApproximation`	`discretization.maximumEntropyWeights`
`GaussianMixtureQuadrature`	`discretization.gaussianMixtureQuadrature`
`NPGQ`	`discretization.dataDrivenGaussianQuadrature`
`CIRpdf`	`discretization.cirTransitionDensity`

The earlier descriptive package names `discreteARWithGaussianMixtureShocks`, `discreteARWithStochasticVolatility`, `discreteDistributionFromMoments`, and `nonparametricGaussianQuadrature` also remain as MATLAB compatibility wrappers.

11 Citation and software responsibility

Validate this research software by checking grid coverage, moment errors, transition-row sums, and sensitivity to state placement. Report the language, revision, routine, grid method, state count, and requested moment count.

Cite Farmer and Toda (2017) for the main method; Tanaka and Toda (2013) and Tanaka and Toda (2015) for the maximum-entropy foundations; and Toda (2021) for data-based nonparametric Gaussian quadrature.

References

Farmer, L. E. and A. A. Toda (2017). "Discretizing nonlinear, non-Gaussian Markov processes with exact conditional moments". *Quantitative Economics* 8.2, 651–683. DOI: 10.3982/QE737.

Tanaka, K. and A. A. Toda (2013). "Discrete approximations of continuous distributions by maximum entropy". *Economics Letters* 118.3, 445–450. DOI: 10.1016/j.econlet.2012.12.020.

Tanaka, K. and A. A. Toda (2015). "Discretizing distributions with exact moments: error estimate and convergence analysis". *SIAM Journal on Numerical Analysis* 53.5, 2158–2177. DOI: 10.1137/140971269.

Toda, A. A. (2021). "Data-based automatic discretization of nonparametric distributions". *Computational Economics* 57, 1217–1235. DOI: 10.1007/s10614-020-10012-6.